Manual del sistema causal local

Descarga · instalación · comprensión · integración · uso · interpretación

Prototipo «Puente» · repositorio `prototipo_ml_causal` · escenario conservado `synthetic/ihdp` · champion `v0009`

Qué es. Un prototipo local y auditable de estimación causal con datos sintéticos. **Qué no es.** Un servicio clínico, laboral o de política pública desplegado en Internet.

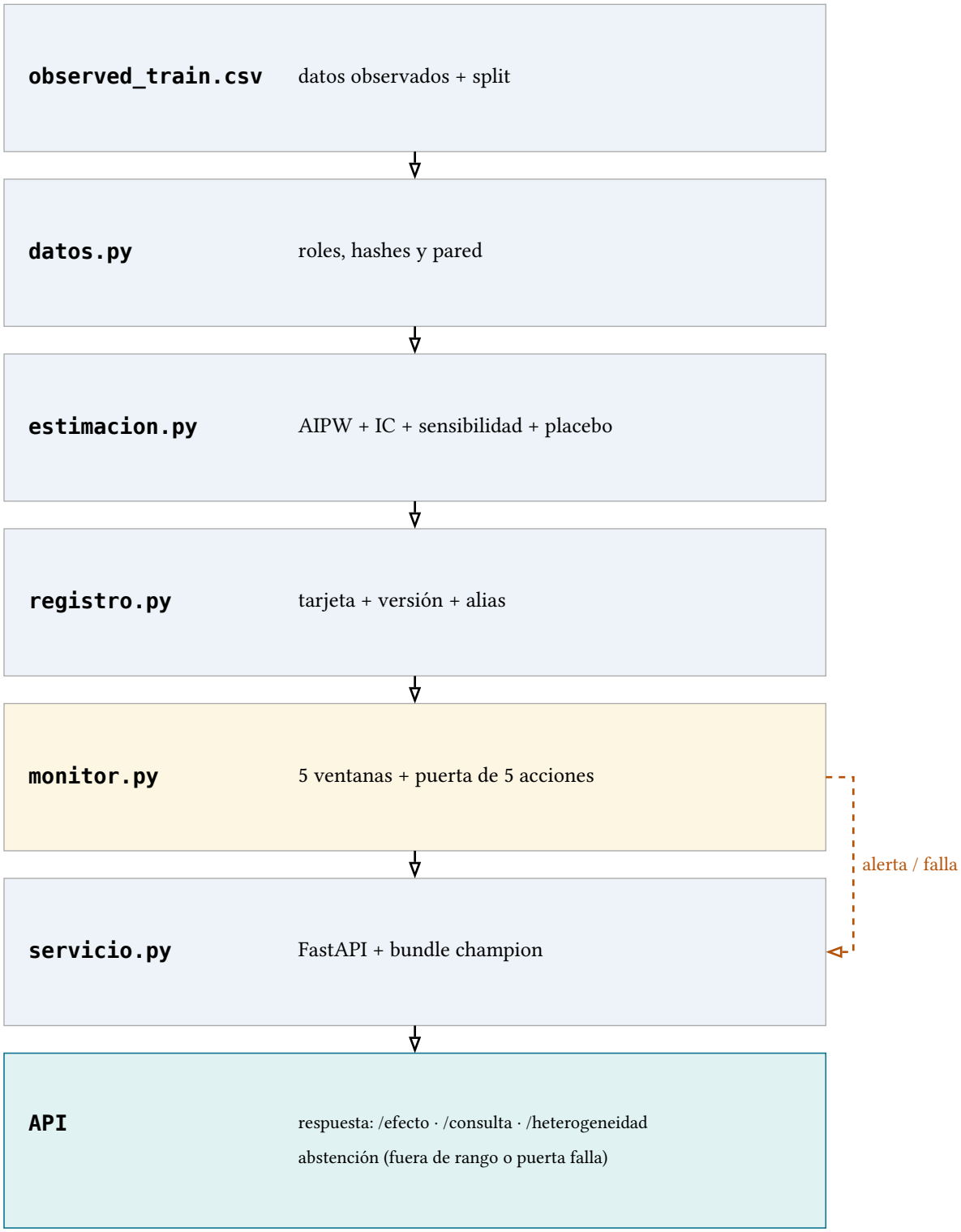


Figura 1: **Del dato al servicio.** La instalación conservada entra directamente en la última etapa: no vuelve a estimar ni promueve versiones. Azul: pipeline; ámbar: puerta; verde: servicio. La flecha discontinua representa la señal del monitor: una puerta activa `falla` bloquea las respuestas, no una instalación ya realizada.

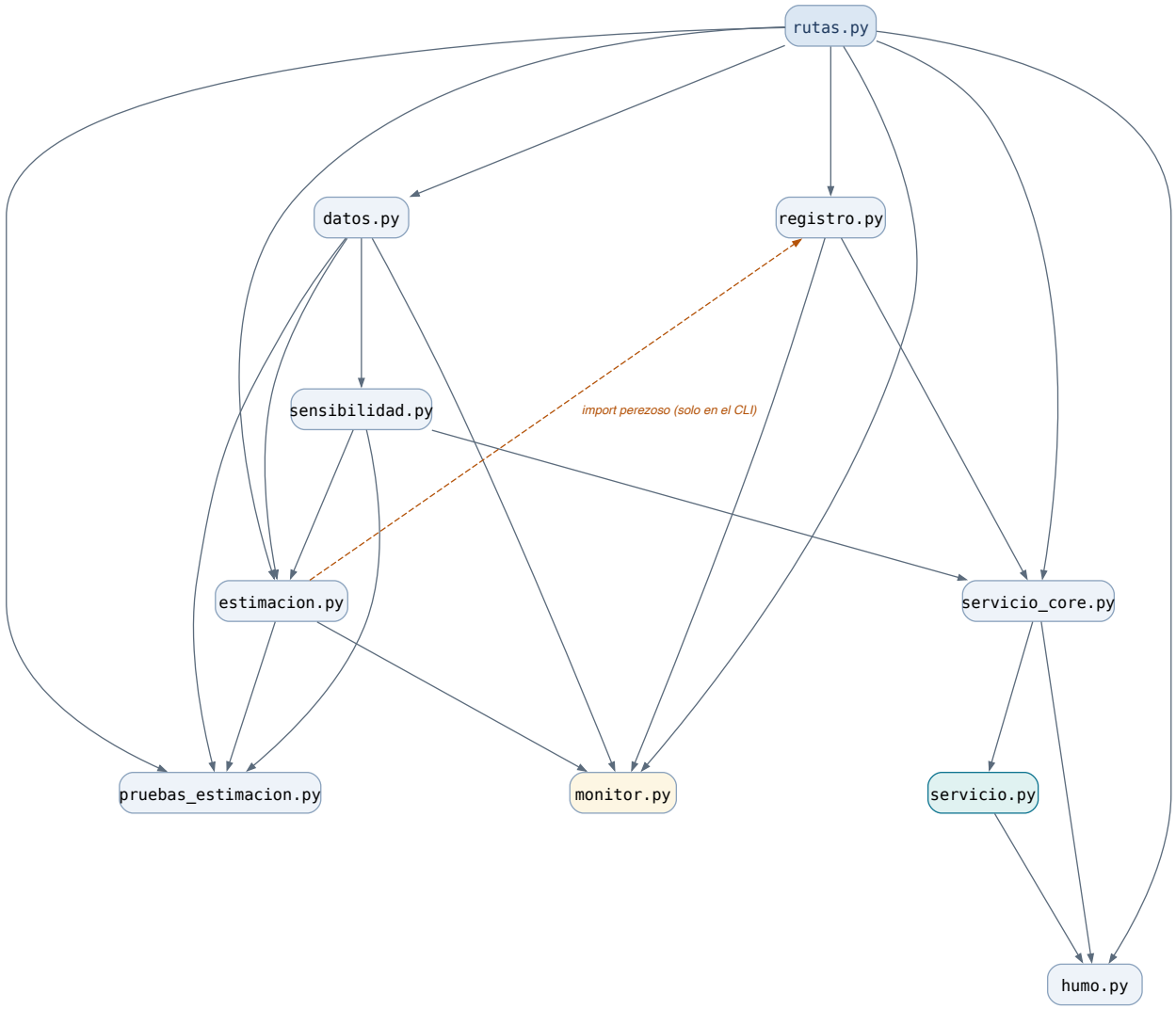


Figura 2: **Dependencias entre módulos.** La flecha va del módulo importado al que importa: `A -> B` significa que `B` hace `import A`. Azul: configuración común; ámbar: monitor; verde: servicio. La flecha discontinua es el import perezoso de `registro.py` desde `estimacion.py`, que ocurre cuando este último se ejecuta como programa. Por otra vía, `servicio_core.py` importa funciones de `registro.py` para resolver y verificar el champion.

Cómo leer el repositorio. Ningún módulo Python orquesta todo el pipeline: el orden está en el `Makefile`. Cada etapa lee y escribe artefactos JSON. `rutas.py` centraliza rutas y escenario; `monitor.py` reutiliza funciones de `estimacion.py`. El servicio carga un champion válido en memoria en su primera consulta y lo reutiliza hasta reiniciarse; una carga fallida no se cachea.

Ruta crítica para evaluarlo. Clona el repositorio, ejecuta el instalador de tu plataforma —sección 2— y abre `/salud` y `/efecto` en `http://127.0.0.1:8000`. Comprueba `version = v0009`, `integridad_valida = true`, `estado_puerta = pasa` y ATE 5,2 [3,9; 6,4]. Si solo necesitas evaluar el servicio conservado, puedes detenerte ahí: no hace falta regenerar nada. Para integrarlo, continúa en las secciones 8–10; para entender los datos y el método, en las 4–7.

Ruta de lectura según tu objetivo.

Objetivo	Secciones
Solo ejecutar y comprobar	1–3: descarga, instalación y validación

Objetivo	Secciones
Entender la inferencia causal	4–7: datos, identificación, estimación y sensibilidad
Consumir o integrar la API	8–10: endpoints, contratos de cliente, puerta y abstención
Operar o auditar	11–13: monitor, evidencia, seguridad y resolución de problemas
Evaluar académicamente	14–16: interpretación, DAG y rúbrica

Contenido

1 Descarga y mapa del repositorio 6
1.1 Obtener el código sin credenciales 6
1.2 Qué contiene cada carpeta 6
2 Instalación y puesta en marcha 6
2.1 Linux o macOS: un comando 6
2.2 Windows sin terminal: doble clic 7
2.3 Docker Compose: alternativa 7
2.4 Qué aparece al terminar 8
2.5 Detener, reiniciar y volver a consultar 8
3 Comprobación y diagnóstico de la instalación 8
3.1 Pruebas que no promueven versiones 8
3.2 Problemas frecuentes 9
4 Datos, variables y pregunta causal 9
4.1 Escenarios y archivos 9
4.2 Origen del dato: tres capas distintas 10
4.3 Anatomía de covariables, tratamiento y resultado 10
4.4 Pregunta, estimando y población 11
5 Identificación y supuestos 11
5.1 Estrategia backdoor y expresión identificada 11
5.2 Qué se puede examinar y qué no 12
6 Estimación, diagnósticos y validación 12
6.1 AIPW con cross-fitting 12
6.2 Resultados que sirve el champion 12
7 Sensibilidad y placebos: interpretación correcta 13
8 Uso del servicio: endpoints y ejemplos 14
8.1 Reglas comunes 14
8.2 ATE poblacional: `GET /efecto` 14
8.3 Media de estrato: `POST /consulta` 15
8.4 Proyección de heterogeneidad: `POST /heterogeneidad` 15
8.5 Estado del sistema: `GET /salud` 16
9 Integración con aplicaciones y con otros datos 16
9.1 Contrato mínimo de un consumidor 16
9.2 HTTP y validación 16
9.3 Unir el sistema a un caso propio 17
10 Puerta, abstención e integridad 17
10.1 Qué bloquea realmente una respuesta 17
10.2 Probar la abstención sin modificar artefactos 17
11 Monitor y regla operativa 18
11.1 Ventanas conservadas 18
11.2 Tarjeta, manifiesto y champion 18
12 Evidencia, mantenimiento y regeneración 19
12.1 Consultar la evidencia existente 19
12.2 Regenerar deliberadamente 19
13 Operación, seguridad y alcance 19
13.1 Validación de esta segunda auditoría 20
14 Interpretación final para lectores no técnicos 21
15 DAG completo de `synthetic/ihdp` 22
16 Reutilización, defensa y rúbrica 23
16.1 Decisiones propias y preparación de la defensa 23
16.2 Estado frente a los criterios 1–6 23

1 Descarga y mapa del repositorio

1.1 Obtener el código sin credenciales

Repositorio público: github.com/jelincovil/prototipo_ml_causal. Con Git instalado:

```
git clone https://github.com/jelincovil/prototipo_ml_causal.git
cd prototipo_ml_causal
```

Sin Git, descarga y descomprime el ZIP: codeload.github.com/jelincovil/prototipo_ml_causal/zip/refs/heads/main. Después abre una terminal dentro de la carpeta descomprimida. El índice de código con enlaces directos está en `AUDITORIA.md`; la descarga incluye código, datos sintéticos, champion y evidencia. No necesitas cuenta de GitHub, token ni datos personales.

1.2 Qué contiene cada carpeta

```
prototipo_ml_causal/
├── setup.sh / setup.ps1          instalación con uv y Python 3.13
├── arrancar.sh / arrancar.ps1    arranque del servicio conservado
├── docker-compose.yml            alternativa Docker, puerto local 8000
├── requirements.txt              dependencias directas fijadas; sin lockfile transitivo
├── healthcheck.py                salud, integridad, puerta y versión para Docker
├── datos.py                      esquema, hashes de splits y pared de verdad
├── estimacion.py                 AIPW, OLS, diagnósticos e identificación
├── sensibilidad.py               E-value, robustez, Rosenbaum, Manski y placebos
├── registro.py                   tarjeta, manifiesto y promoción con comprobaciones
├── monitor.py                    ventanas, deriva y puerta
├── servicio_core.py              bundle, integridad y contratos de respuesta
├── servicio.py                   endpoints FastAPI y formulario web
├── pruebas_estimacion.py         fórmulas e invariantes
├── humo.py                       contratos de servicio y abstenciones
├── artifacts/                    champion v0009 y versiones históricas
├── evidencia/                    comandos, salidas y hashes del cierre
├── data_benchmark_causal/         escenarios, metadatos y este manual
└── docs/                          pregunta, guías, bitácora e histórico
```

La API **no entrena con peticiones**, no ofrece carga de CSV y no escribe modelos al contestar. Consume únicamente el bundle referenciado por `artifacts/alias_champion.json`. `SIC_ESCENARIO=synthetic/ihdp` fija el contexto de datos y de las pruebas de humo; no es un selector remoto de modelos. Para cambiar de champion se usa el flujo controlado de registro y promoción, se verifica el estado resultante y se reinicia el servicio; no se cambia por una petición HTTP.

2 Instalación y puesta en marcha

2.1 Linux o macOS: un comando

Desde la raíz del clon:

```
bash setup.sh && bash arrancar.sh
```

El primer script instala `uv` mediante su instalador oficial si falta, instala Python 3.13, **recrea `.venv`** e instala `requirements.txt`. Repetirlo deja un entorno coherente, pero descarta paquetes extra o personalizaciones del entorno anterior. El segundo activa ese entorno, fija `synthetic/ihdp` y ejecuta Uvicorn en `127.0.0.1:8000`. Requerimientos del sistema: Bash y `curl` o `wget`; Git solo es necesario para clonar.

`requirements.txt` fija las dependencias directas, no todas las transitivas. `uv pip check` comprueba compatibilidad, pero no sustituye un lockfile completo ni una verificación de hashes de paquetes.

Para instalar y arrancar por separado:

```
bash setup.sh
bash arrancar.sh
```

La instalación es repetible: puedes volver a ejecutar `setup.sh`, teniendo en cuenta que recrea el entorno. El arranque no es repetible en el mismo puerto: no ejecutes dos copias simultáneas del servicio.

2.2 Windows sin terminal: doble clic

La vía más simple no exige abrir Terminal, PowerShell ni CMD:

1. Descarga **PuenteCausal.exe** del release oficial.
2. Haz doble clic y pulsa **Instalar / verificar**.
3. Pulsa **Iniciar servicio** y después **Abrir ATE**.

El lanzador instala el código de la versión publicada, `uv` 0.13.8 y Python 3.13 en `%LOCALAPPDATA%\PuenteCausal`, no en `Documents`. Reutiliza un entorno válido al repetir la instalación. **Reparar entorno** pide confirmación, conserva el anterior en `respaldo/` y crea uno nuevo. Ejecuta Uvicorn directamente con `synthetic/ihdp`; no activa una consola y no regenera estimaciones, artefactos ni el champion `v0008` fijado en ese release.

Los botones **Detener**, **Abrir /docs** y **Abrir registros** permiten operar y diagnosticar sin comandos. Cerrar el lanzador detiene el servicio que haya iniciado. Para desinstalar, detén el servicio y borra `PuenteCausal` dentro de `%LOCALAPPDATA%` desde el Explorador; luego borra el `.exe` descargado. Los registros permanecen en el equipo y no se envían a Internet.

El binario no tiene firma comercial, así que SmartScreen puede advertir. El release también incluye `PuenteCausal.exe.sha256`. La autoevaluación de compilación no equivale a validar la interacción gráfica en cada PC. Consulta `instalador_windows/LEEME.md` para procedencia, descargas externas, privacidad y diagnóstico de permisos o antivirus.

2.2.1 Alternativa: Windows PowerShell

Desde la raíz del clon, en PowerShell:

```
powershell -ExecutionPolicy Bypass -File setup.ps1
if ($LASTEXITCODE -eq 0) {
    powershell -ExecutionPolicy Bypass -File arrancar.ps1
}
```

La condición evita arrancar si falla `setup` y funciona también en Windows PowerShell 5.1.

Por separado:

```
powershell -ExecutionPolicy Bypass -File setup.ps1
powershell -ExecutionPolicy Bypass -File arrancar.ps1
```

Esta alternativa sí **recrea `.venv`**. Ha recibido revisión estática, no una validación completa del flujo en Windows nativo. Si `uv` queda fuera de `PATH`, cierra y abre PowerShell y repite el comando. Para los ejemplos HTTP usa `curl.exe`, no el alias histórico `curl`.

2.3 Docker Compose: alternativa

Con Docker Engine y el plugin Compose activos:

```
docker compose up
```

La imagen usa el `Dockerfile` existente, Python 3.13 y solo las dependencias necesarias para servir JSON. Copia `artifacts/`, no necesita los CSV y ejecuta como usuario no root. Compose publica `127.0.0.1:8000:8000` y fija `synthetic/ihdp`. Para reconstruir tras un cambio deliberado del código usa `docker compose up --build`. Detén con Ctrl-C; para retirar el contenedor, `docker compose down`. La imagen base no tiene digest fijado y Compose no fue ejecutado en esta auditoría; sí se probaron `docker build` y `docker run`.

2.4 Qué aparece al terminar

Abre estas direcciones en el navegador:

URL	Para qué sirve
http://127.0.0.1:8000/	Formulario web de heterogeneidad
http://127.0.0.1:8000/efecto	ATE del champion, intervalo y diagnósticos
http://127.0.0.1:8000/salud	Versión, puerta, integridad y resúmenes
http://127.0.0.1:8000/docs	Swagger UI y prueba interactiva de rutas
http://127.0.0.1:8000/openapi.json	Esquema OpenAPI generado por FastAPI

Con los artefactos conservados debes ver `version = v0009`, `estado_puerta = pasa`, `integridad_valida = true` y, en `/efecto`, `efecto = 5.2` con `ic95 = [3.9, 6.4]`. Las respuestas pueden contener más campos que los fragmentos mostrados en este manual.

La instalación no regenera resultados. Estos comandos no ejecutan `make demo` ni `make evidencia`, no modifican `artifacts/` y no promueven un champion. Publican el bundle conservado. No ejecutes los comandos de regeneración de la sección 12 solo para arrancar el servicio.

2.5 Detener, reiniciar y volver a consultar

Pulsa **Ctrl-C** en la terminal donde corre Uvicorn o Docker Compose. Para volver a levantarlo usa solo `bash arrancar.sh`, `powershell -ExecutionPolicy Bypass -File arrancar.ps1` o `docker compose up`, según el método elegido. Repite `setup` únicamente si falta el entorno o cambiaron las dependencias; hacerlo descarta personalizaciones de `.venv`. Un bundle válido se carga en la primera consulta y queda en memoria hasta reiniciar; una carga fallida no se cachea. Reinicia tras una actualización autorizada del champion, del manifiesto o del monitor cuando ya exista una caché válida.

3 Comprobación y diagnóstico de la instalación

3.1 Pruebas que no promueven versiones

En Linux/macOS, sin activar `.venv` ni necesitar `make`:

```
SIC_ESCENARIO=synthetic/ihdp .venv/bin/python datos.py
SIC_ESCENARIO=synthetic/ihdp .venv/bin/python pruebas_estimacion.py
SIC_ESCENARIO=synthetic/ihdp .venv/bin/python humo.py
shasum -a 256 -c evidencia/SHA256SUMS
# Alternativa habitual en Linux:
sha256sum -c evidencia/SHA256SUMS
```

Equivalentes directos en PowerShell:

```
$env:SIC_ESCENARIO = "synthetic/ihdp"
.\.venv\Scripts\python.exe datos.py
.\.venv\Scripts\python.exe pruebas_estimacion.py
.\.venv\Scripts\python.exe humo.py
```

Con `make` y el entorno activado puedes usar `make verificar`, `make pruebas` y `make humo`. Resultado esperado: `PRUEBAS: OK`, `HUMO: OK` y ningún hash incorrecto. En Windows, `Get-FileHash -Algorithm SHA256 <archivo>` contrasta un archivo individual con `evidencia/SHA256SUMS`; no procesa toda la lista. Las pruebas de humo llaman a las funciones del servicio y comprueban sus contratos; no necesitan otro Uvicorn en el puerto 8000.

3.2 Problemas frecuentes

Síntoma	Comprobación y solución
`uv: command not found`	Abre una terminal nueva y repite `setup`; comprueba que el directorio de instalación de `uv` está en `PATH`.
Falta `.venv` al arrancar	Ejecuta primero el instalador del sistema operativo. El script de arranque deliberadamente no crea el entorno.
`Address already in use`	Ya hay un proceso en 8000. Detén el servicio anterior o identifica qué programa ocupa el puerto; no promuevas otro champion para resolverlo.
No abre `127.0.0.1:8000`	Confirma que la terminal sigue abierta y que la URL usa `http`, no `https`. Revisa firewall local y los mensajes de Uvicorn.
`docker compose` no existe	El Engine no basta: instala o habilita el plugin Compose, o utiliza los scripts con `uv`.
Error de hash o integridad	No estimes por encima. Restaura el bundle desde una copia limpia y revisa alias, `manifest.json` y archivos de `v0009`; el servicio debe abstenerse.
La respuesta no coincide	Lee `/salud`: versión, integridad y puerta. Los ejemplos de este manual corresponden específicamente al champion conservado `v0009`.

4 Datos, variables y pregunta causal

4.1 Escenarios y archivos

Hay dos escenarios **sintéticos** con la misma estructura; el champion sirve IHDP simulado.

Dato	`synthetic/ihdp`	`synthetic/jobs`
Inspiración	Hill (2011), JCGS 20(1)	LaLonde (1986), AER 76(4)
Unidades (total; train / validation / test)	747 (448 / 149 / 150)	1722 (1033 / 344 / 345)
Covariables	25 continuas simuladas	8 (`age` … `re75`)
Tratados en train	191 de 448 (42,6 %)	262 de 1033 (25,4 %)
Columna de tratamiento	`treatment`	`treatment`
Columna de resultado	`outcome`: puntaje cognitivo a 3,5 años	`re78`: ingresos de 1978 (USD)

Cada carpeta bajo `datasets/synthetic/` contiene:

- `observed.csv` y `observed_{train,validation,test}.csv`: pool y particiones estratificadas 60/20/20, semilla `20240701`. `splits.json` registra tamaños y hashes SHA-256; `datos.py` los recalcula.
- `metadata.json`: ficha del escenario; `identification.json`: conjunto de ajuste declarado. Son anotaciones del benchmark, no una prueba empírica independiente.
- `interventions.json`: referencias agregadas `ate`, `att` y resumen de `tau`. Se reserva para la comparación final; no alimenta estimación, selección ni calibración.
- No hay resultados potenciales individuales observados ni verdad causal por fila. Aunque exista una referencia agregada, no sirve como etiqueta supervisada individual.

Datos y autorización. El paquete se genera por simulación y no contiene registros personales. Si se adapta a datos propios, la autorización, minimización y protección de esos datos deben resolverse antes de incluirlos; no basta con sustituir el CSV.

4.2 Origen del dato: tres capas distintas

«IHDP» puede nombrar un ensayo real, un benchmark semi-sintético y este escenario sintético. No deben mezclarse sus resultados.

Capa 1 — ensayo real. El *Infant Health and Development Program* fue un ensayo aleatorizado multicéntrico en ocho hospitales de EE. UU., con intervención entre 1985 y 1988. Participaron lactantes de bajo peso y prematurez: de 1.302 elegibles, 985 se aleatorizaron, 377 a intervención y 608 a seguimiento. La intervención combinó visitas domiciliares, un centro de desarrollo infantil entre los 12 y 36 meses y grupos de apoyo para padres. El resultado primario fue el CI de Stanford–Binet a los 36 meses. Esas cifras pertenecen al ensayo, **no** al CSV de este repositorio.

Capa 2 — benchmark de Hill (2011). Hill conservó covariables y tratamiento del ensayo, simuló resultados y filtró tratados cuya madre no era blanca, dejando 139 tratados y 608 controles (747 unidades). Ese filtrado produce falta de solapamiento en ciertos grupos: el ATE del benchmark canónico no debe darse por identificado automáticamente; se suelen estudiar efectos en los tratados. Este problema de diseño no se resuelve cambiando de estimador.

Capa 3 — paquete actual. `synthetic/ihdp` no contiene registros del ensayo ni la réplica canónica del benchmark: `metadata.json` declara `SYNTHETIC - generated from scratch using formula-based simulation (Hill 2011 style)`. Conserva el tamaño 747 y nombres de variables como referencia semántica, pero simula covariables continuas, tratamiento logístico en `X`, resultado lineal con heterogeneidad y ruido independiente. El paquete declara semilla 400, pero esa semilla **no basta para reconstruir exactamente los CSV publicados**: falta el generador DGP completo con su orden de aleatorización y versiones. `cobertura.py` simula otro DGP y no es el generador de IHDP. Los CSV incluidos y sus hashes son la referencia comprobable. Sus números prueban la cadena metodológica, no la eficacia del programa real.

Rasgo	**`synthetic/ihdp`**
Unidades	747 en total: 448 en `train`, 149 en `validation`, 150 en `test`
Covariables	25, todas continuas y simuladas
Tratamiento: `treatment`	0/1. 1 = programa de intervención temprana (visitas domiciliares, centro infantil y grupos de padres hasta los 36 meses); 0 = seguimiento. La etiqueta representa el brazo; no se simulan visitas ni dosis.
Resultado: `outcome`	Puntaje cognitivo a los 3,5 años, continuo, posterior al tratamiento; mayor es mejor. La unidad es puntos del test, no dólares ni probabilidad.
Referencia agregada	ATE 3,92 y ATT 4,15 en `interventions.json`; solo comparación final

Fuentes de contexto. Hill (2011), *Journal of Computational and Graphical Statistics* 20(1), 217–240, doi:10.1198/jcgs.2010.08162; Infant Health and Development Program (1990), *JAMA* 263(22), 3035–3042; Brooks-Gunn y cols. (1994), *JAMA* 272(16), 1257–1262. Estas fuentes explican el origen conceptual; la evidencia numérica del prototipo está en `artifacts/` y `evidencia/`, no derivada de los participantes reales.

4.3 Anatomía de covariables, tratamiento y resultado

El escenario tiene 747 filas y 27 columnas: 25 covariables, tratamiento y resultado. Los nombres clínicos ayudan a leer el DAG, pero los valores de las 25 covariables son simulados y continuos: no representan mediciones clínicas ni indicadores binarios reales.

Bloque	**Variables**	**Papel en el diseño**
Neonatal (5)	`bw`, `b.head`, `preterm`, `birth.o`, `nnhealth`	Peso, medidas y condición al nacer
Del niño (2)	`sex`, `twin`	Características del niño
Materno (6)	`momage`, `prenatal`, `cig`, `booze`, `drugs`, `first`	Edad, conducta y cuidado prenatal

Bloque	Variables	Papel en el diseño
Socioeconómico (5)	`mom.lths`, `mom.hs`, `mom.scoll`, `b.marr`, `work.dur`	Educación, hogar y trabajo
Sitio (7)	`ark`, `ein`, `har`, `mia`, `pen`, `tex`, `was`	Etiquetas de centro o contexto

Todas influyen en `D` y `Y` por construcción del DGP y componen el conjunto de ajuste. El tratamiento se genera con un modelo logístico en esas covariables; el resultado depende de ellas y de `treatment`. Esta arquitectura introduce confusión deliberada para que el ejercicio tenga algo que corregir.

La diferencia cruda media en `train` es 10,3 puntos (46,6 frente a 57,0), pero mezcla efecto y composición. Los comparadores ajustados son OLS 4,8 [3,8; 5,8] y AIPW 5,2 [3,9; 6,4]. La coincidencia aproximada de ajustados es estabilidad entre estimadores, no prueba de ignorabilidad. Sobre una DE aproximada de 15 puntos, 5,2 equivale a unas 0,34 DE.

4.4 Pregunta, estimando y población

Pregunta de decisión. ¿Debe ampliarse el programa de intervención temprana a la población del estudio para mejorar el puntaje cognitivo a los 3,5 años?

El estimando declarado antes de observar los efectos es el **efecto promedio del tratamiento** (ATE):

$$\tau = E[Y(1) - Y(0)]$$

- **Población objetivo:** las 448 unidades de `observed_train.csv` del escenario. El AIPW promedia scores sobre esa muestra; no es una estimación de toda la infancia real.
- **Unidad:** puntos del test cognitivo. `/efecto` devuelve diferencia media `Y(1) - Y(0)`, no el puntaje de un niño ni la probabilidad de mejorar.
- **Cobertura:** el IC del 95 % expresa incertidumbre del efecto promedio bajo el modelo y procedimiento declarados. No es un intervalo de predicción individual ni de un CATE.
- **ATE no es ATT.** «En quienes recibieron el programa» formula otro estimando. Las referencias 3,92 y 4,15 se distinguen precisamente por eso.
- **Decisión:** un ATE positivo no implica por sí solo ampliar un programa: también importan validez externa, costes, riesgos y comparación con alternativas.

5 Identificación y supuestos

5.1 Estrategia backdoor y expresión identificada

La estrategia es ajuste por las 25 covariables pre-tratamiento. `identificacion_dowhy()` construye el DAG y consulta DoWhy con `method_name="default"`; en el champion se obtiene el criterio **backdoor**, el conjunto coincide con `valid_adjustment_sets` y `identificacion_pasa = true`. DoWhy comprueba coherencia algebraica con el grafo introducido: no descubre ni valida el DAG a partir de los datos. No hay instrumento, panel pre/post o umbral que justifique IV, diferencias en diferencias o RD.

Los supuestos causales centrales son:

- **A1, ignorabilidad condicional:** $Y(0), Y(1)$ independientes de D dado X.
- **A2, positividad:** $0 < e(X) < 1$ en la población objetivo.
- **A3, consistencia / SUTVA:** $Y = Y(D)$, tratamiento bien definido y sin interferencia.

Con A1–A3 se identifica cada media potencial mediante estandarización:

$$E[Y(d)] = E_{X[E[Y \mid X, D=d]]}, \quad d \in \{0, 1\}$$

Por tanto:

$$\tau = E_{X[E[Y \mid X, D=1] - E[Y \mid X, D=0]]}$$

La identificabilidad procede del modelo causal y los supuestos, no del algoritmo AIPW ni de que DoWhy devuelva `pasa = true`. En este DGP el tratamiento se asigna por `X` y no se simula un confusor latente; en datos

reales, ignorabilidad seguiría siendo una defensa de diseño y no un chequeo que los datos observados puedan certificar.

5.2 Qué se puede examinar y qué no

Supuesto	Clase	Evidencia y consecuencia si falla
Ignorabilidad condicional	`no_observable`	Sostenida por el DGP sintético. Balance, placebos y sensibilidad exploran consecuencias; ninguno elimina la posibilidad de confusión en otro contexto. Si falla, el contraste puede quedar confundido y el ATE no está identificado por este ajuste.
Positividad / soporte común	`observable`	OOF antes de trimming: propensión [0,043; 0,916], solape 0,99777 y ESS 332,2076/448 = 0,7415; se recorta 1 unidad de 448. Diagnóstico muestral, no prueba para todo vector nuevo. Si A2 falla, restringir población, acotar o abstenerse.
Consistencia / SUTVA	`no_observable`	El DGP tiene unidades independientes, un resultado por tratamiento y ausencia de spillovers. En un programa real, versiones e interferencia pueden cambiar el estimando; no existe un test directo en este paquete.
Covariables pre--tratamiento	`observable` según diseño	Lo declaran los metadatos. `score_post` se deriva de `Y` y actúa como demostración de fuga: vacía el coeficiente. No es covariable legítima ni placebo.

Rangos no significan soporte multivariable. La abstención por rango revisa marginales conocidos, no garantiza que una combinación completa tenga controles y tratados comparables. De la misma forma, el buen balance ponderado no prueba ignorabilidad.

6 Estimación, diagnósticos y validación

6.1 AIPW con cross-fitting

El estimador principal es AIPW con $K = 5$ y semilla `4242`; los modelos auxiliares se ajustan fuera del pliegue que predicen. El resultado por brazo usa OLS y la propensión, regresión logística. El recorte de propensión declarado es [0,05; 0,95].

$$\psi_i = \hat{\mu}_1(X_i) - \hat{\mu}_0(X_i) + \frac{D_i(Y_i - \hat{\mu}_1(X_i))}{\hat{e}(X_i)} - \frac{(1 - D_i)(Y_i - \hat{\mu}_0(X_i))}{1 - \hat{e}(X_i)}$$

$$\hat{\tau} = \frac{1}{n}\sum_{i=1}^{n} \psi_i, \quad \hat{\text{se}} = \frac{\text{sd}(\psi)}{\sqrt{n}}$$

La doble robustez requiere regularidad y al menos una pierna bien especificada, además de los supuestos causales. Cross-fitting evita que una unidad contribuya a su propia predicción auxiliar; no arregla modelos lineales incorrectos ni confusión no observada.

6.2 Resultados que sirve el champion

Cantidad	`synthetic/ihdp · v0009`
Diferencia cruda	10,3 puntos; IC aprox. [7,60; 13,05]
OLS ajustado (comparador)	4,8 puntos; IC [3,8; 5,8]
ATE AIPW	**5,2 puntos; IC [3,9; 6,4]**
ESS OOF / n	332,2076 / 448 = 0,7415
Propensión OOF antes de trimming	[0,043; 0,916]
Solape OOF en [0,05; 0,95]	0,99777 (aprox. 99,8 %)
Unidades recortadas	1/448 (0,2232 %), un control; ningún tratado

Cantidad	`synthetic/ihdp · v0009`
SMD máximo: sin ponderar / ponderado	0,43 / 0,11
R^2 de la proyección de heterogeneidad	0,095; discriminación baja

Validación por particiones. AIPW se reestima **independientemente** en `train`, `validation` y `test`: no es el holdout predictivo de un único modelo entrenado en `train`. `train` define la estimación primaria; las demás son chequeos de estabilidad con menos unidades, no nuevas pruebas de identificación:

Partición	n	ATE AIPW	IC 95 %
train	448	5,16	[3,93; 6,38]
validation	149	1,93	[−2,45; 6,31]
test	150	1,63	[−3,91; 7,18]

Los IC se solapan, pero los efectos fuera de `train` son menores e imprecisos: no declaran estabilidad precisa. El estrato predeclarado por `bw` en la mediana produce 7,2 [5,5; 8,8] en `bw` alto y 3,2 [1,4; 4,9] en `bw` bajo, 224 unidades por grupo. Son medias de scores AIPW en estratos; no constituyen un CATE flexible validado.

Cobertura Monte Carlo. Con un DGP ilustrativo propio —no IHDP ni Jobs—, $n = 500$, $R = 100$ y semilla `20261111`, las coberturas IC95 fueron AIPW 0,97 (error MC 0,01706), OLS 0,96 (0,01960) e ingenuo 0,00 (0). Es evidencia limitada a ese DGP y no demuestra calibración exacta ni validez en datos reales.

7 Sensibilidad y placebos: interpretación correcta

Medida	Cifra de referencia	Qué responde y qué no
E-value aproximado	2,07; IC 1,85	Fuerza de asociación aproximada con `D` y `Y`, condicionada en `X`, necesaria para explicar el punto o el límite del IC. Para resultado continuo usa efecto estandarizado: no es una RR literal, ni cubre mala especificación, positividad o SUTVA.
Valor de robustez OLS	37,4 %	Fracción del R^2 parcial residual de `D` y `Y` que un confusor debería explicar para llevar OLS a cero. Es exacto para ese OLS, no para el AIPW.
Rosenbaum Γ	2,21; 179 pares (`v0009`)	Wilcoxon de pares 1:1, emparejados por propensión OOF con caliper 0,2·DE(logit). Acota el p-valor del contraste emparejado bajo odds ocultas de asignación; no acota el ATE AIPW. Depende del emparejamiento y usa aproximación normal.
Cotas de Manski	[−37,66; 48,48]	Sin ignorabilidad no hay identificación puntual; las cotas incluyen cero. Ilustración con `Y` observado en [10,41; 96,54]; extremos muestrales no garantizan soporte real.
Placebo de tratamiento	pasa; p = 0,005	200 permutaciones, pliegues fijos: media −0,04, DE 0,62, IC [−1,42; 1,06]; el ATE observado es extremo. Examina calibración e implementación, no ausencia de confusión.
Placebo de resultado	pasa; 0,04 [−0,23; 0,32]	Pseudo-resultado construido permutando `momage`; umbral material 0,10. Un resultado cercano a cero es coherente con calibración, no una prueba de ignorabilidad.

Si falla un placebo o control negativo, la acción es `revisar_identificacion`. No debe «arreglarse» el número eligiendo otra semilla, filtrando hasta que el placebo pase o leyendo el E-value como certificado de validez.

8 Uso del servicio: endpoints y ejemplos

8.1 Reglas comunes

Base local: `http://127.0.0.1:8000`. Las peticiones POST reciben **directamente** un objeto JSON de perfil, sin envoltura `{"perfil": ...}`. Usa números JSON finitos, no texto, `null`, listas ni `NaN`. Las claves extra se ignoran. Los rangos son inclusivos y provienen del champion; los datos sintéticos no están restringidos a 0/1 aunque el nombre original de una covariable fuese binario.

En `/efecto`, `/consulta`, `/heterogeneidad` y `/efecto_individual`, el query parameter `formato=texto` (también `txt` o `plain`) devuelve texto plano. Otro valor devuelve JSON. `/salud` siempre devuelve JSON. La documentación interactiva `/docs` es útil para explorar, pero el campo `perfil: dict` no enumera las covariables: el contrato siguiente sí lo hace.

Método y ruta	Entrada	Salida
`GET /efecto`	ninguna	ATE poblacional, IC, identificación, sensibilidad y placebos
`POST /consulta`	`bw` obligatorio; demás opcionales	Media de uno de dos estratos
`POST /heterogeneidad`	las 25 covariables	`proyeccion`, IC condicional y `diagnosticos_globales` separado
`POST /efecto_individual`	igual que heterogeneidad	Alias deprecado; no otro estimador
`GET /salud`	ninguna	Versión, puerta, integridad y resúmenes
`GET /`	ninguna	Formulario HTML
`GET /calcular`	covariables query	Informe separado: perfil y diagnósticos globales

8.2 ATE poblacional: `GET /efecto`

```
curl -sS http://127.0.0.1:8000/efecto
curl -sS 'http://127.0.0.1:8000/efecto?formato=texto'
```

Fragmento simplificado de la respuesta conservada (la real incluye más diagnósticos):

```
{
  "estado": "respuesta",
  "tipo": "poblacional",
  "efecto": 5.2,
  "unidad": "puntos de test cognitivo",
  "ic95": [3.9, 6.4],
  "estado_puerta": "pasa",
  "version": "v0009"
}
```

Lectura: en la población sintética de entrenamiento, el programa se asocia a un efecto causal promedio estimado de 5,2 puntos bajo los supuestos declarados. El intervalo no cubre efectos individuales. `identificacion` comunica backdoor/DoWhy. En `v0009`, `sensibilidad` incluye E-value, robustez OLS, `cotas_manski` y `rosenbaum`: estas dos últimas medidas traen `lectura` y `limites`; ambos placebos traen `lectura` y `que_no_prueba`. El formato texto incluye las mismas interpretaciones. Son campos para revisar, no adornos para sustituir el estimando.

8.3 Media de estrato: `POST /consulta`

```
curl -sS -X POST http://127.0.0.1:8000/consulta \
  -H 'Content-Type: application/json' \
  -d '{"bw":1}'
```

En PowerShell, esta alternativa evita diferencias de comillas y continuación de línea:

```
$perfil = @{ bw = 1 } | ConvertTo-Json
Invoke-RestMethod -Uri http://127.0.0.1:8000/consulta `
  -Method Post -ContentType "application/json" -Body $perfil
```

Para otras rutas POST se usa el mismo patrón con el objeto completo. Si prefieres `curl.exe`, recuerda que la barra `\` de los ejemplos Unix **no** es continuación de línea en PowerShell. `bw = 1` está dentro del rango activo [−3,1332; 3,0532] y por encima de la mediana guardada (aprox. −0,01335), de modo que selecciona `bw` alto. Fragmento esperado:

```
{
  "estado": "respuesta",
  "tipo": "estrato",
  "efecto": 7.2,
  "ic95": [5.5, 8.8],
  "estrato": {"variable": "bw", "valor": 1, "n": 224},
  "estado_puerta": "pasa",
  "version": "v0009"
}
```

Solo `bw` cambia el estrato. Otras covariables presentes se validan contra su rango pero no alteran el resultado. `{}` produce HTTP 200 con abstención porque falta `bw`. Esta ruta no calcula una predicción individual y no interpola un CATE. Su `sensibilidad` es la del estrato cuando es aplicable; su `placebo` remite al diagnóstico poblacional.

8.4 Proyección de heterogeneidad: `POST /heterogeneidad`

Requiere las 25 covariables. Como todas las escalas del escenario admiten cero, este perfil es un ejemplo estructuralmente válido para `v0009`:

```
curl -sS -X POST http://127.0.0.1:8000/heterogeneidad \
  -H 'Content-Type: application/json' \
  -d '{
    "bw": 0, "b.head": 0, "preterm": 0, "birth.o": 0, "nnhealth": 0,
    "momage": 0, "sex": 0, "twin": 0, "b.marr": 0, "mom.lths": 0,
    "mom.hs": 0, "mom.scoll": 0, "cig": 0, "first": 0, "booze": 0,
    "drugs": 0, "work.dur": 0, "prenatal": 0, "ark": 0, "ein": 0,
    "har": 0, "mia": 0, "pen": 0, "tex": 0, "was": 0
  }'
```

La respuesta tiene `estado = respuesta`, `tipo = heterogeneidad_estimada`, `proyeccion`, `ic95`, `ajuste_r2 = 0.095`, `confianza_discriminacion = baja` y `n_entrenamiento = 448`. El **objeto principal del perfil** no tiene el campo `efecto`; `sensibilidad.aplicable` y `placebo.aplicable` son `false`. El bloque `identificacion` copiado del bundle se refiere al ATE poblacional, no identifica ese perfil.

El bloque separado `diagnosticos_globales` sí contiene el ATE e IC globales, sensibilidad —incluidos Rosenbaum y Manski—, placebos, identificación y sus lecturas. Es referencia poblacional: no pertenece al perfil ni valida su proyección. El formato texto y `/calcular` muestran **1. Proyección del perfil** y **2. Diagnósticos globales del ATE** en secciones independientes; no deben combinarse sus inferencias.

Interpretación. Es una proyección lineal del score AIPW con IC condicional al ajuste (`v'Cov(beta)v`). No es un CATE identificado, un efecto individual, un intervalo de predicción ni una recomendación de tratar a una persona. Valores ausentes o fuera de rango causan abstención; un valor con $|z| > 2{,}5$ añade advertencia

aunque esté dentro del rango marginal. `/efecto_individual` tiene el mismo cuerpo y núcleo, más avisos de deprecación; para integraciones nuevas usa `/heterogeneidad`.

8.5 Estado del sistema: `GET /salud`

```
curl -sS http://127.0.0.1:8000/salud
```

Revisa `version`, `estado_puerta`, `integridad_valida`, `diagnosticos_positividad`, `comparacion_estimadores`, `identificacion`, `sensibilidad` y `placebo`. Esta ruta es un informe de estado, no un chequeo binario de autorización: HTTP 200 puede coexistir con `estado_puerta = falla`; con integridad rota devuelve `estado = abstencion`. Un cliente debe leer el cuerpo y aplicar las reglas de las secciones 9 y 10, no considerar saludable todo lo que responda 200. En cambio, el `HEALTHCHECK` del Dockerfile sí exige `estado = ok`, `integridad_valida = true`, `estado_puerta = pasa` y la versión esperada (`v0009` por defecto); un HTTP 200 sin esas condiciones deja el contenedor no saludable. Además, `sensibilidad` ofrece `rosenbaum_interpretacion`, `rosenbaum_limites`, `manski_interpretacion` y `manski_limites`; cada placebo resume `interpretacion` y `que_no_prueba`.

9 Integración con aplicaciones y con otros datos

9.1 Contrato mínimo de un consumidor

Una integración correcta no busca siempre `efecto`. Debe seguir esta secuencia:

1. Comprobar transporte y código HTTP. Un 404/405/422/500 no es una abstención causal.
2. Interpretar `Content-Type`: si se pide texto, no intentar analizarlo como JSON.
3. Leer `estado`. Solo `respuesta` permite leer una estimación; `abstencion` conserva `motivo` y no debe sustituirse por cero, por el último número ni por un promedio.
4. Leer `tipo` y seleccionar el campo: `poblacional` y `estrato` usan `efecto`; `heterogeneidad_estimada` usa `proyeccion`.
5. Propagar `version`, `estado_puerta`, `ic95`, `limitaciones` y advertencias al registro y a la interfaz. No ocultar `alerta`, `reentrenar` o `revisar_identificacion`.
6. Recalcular ninguna cifra en el cliente. La API publica el cálculo del bundle; la UI da contexto, unidad, población y significado del intervalo.

Situación	Conducta del cliente
`estado = respuesta`, `tipo = poblacional`	Mostrar ATE e IC como promedio; no personalizarlo
`estado = respuesta`, `tipo = estrato`	Mostrar variable, estrato, n e IC; no llamarlo perfil individual
`estado = respuesta`, `tipo = heterogeneidad_estimada`	Mostrar `proyeccion`, R^2 y advertencias; nunca afirmar efecto individual
`estado = abstencion`	No mostrar número; enseñar motivo, versión y puerta
Puerta distinta de `pasa`	Mantener el estado visible y escalar según la política de la sección 10

9.2 HTTP y validación

Código	Significado en este prototipo
200	Puede ser respuesta o abstención. Hay que inspeccionar el cuerpo.
404	Ruta inexistente.
405	Método incorrecto: por ejemplo `GET /consulta` o `POST /efecto`.
422	Cuerpo ausente, JSON inválido o cuerpo que no es objeto, según validación de FastAPI.
500	Excepción interna no capturada. Valores no convertibles con `float()` —por ejemplo `{"bw":"alto"}`— pueden llegar aquí; los clientes deben enviar números finitos.

La falta de covariables en un objeto JSON válido suele devolver **200 + abstención**, no 422. Una API de producción estricta añadiría esquemas Pydantic por endpoint y manejo uniforme de errores; ese endurecimiento no se presenta como ya implementado.

9.3 Unir el sistema a un caso propio

El servicio no acepta un CSV nuevo ni reestima online. Adaptar el caso exige repetir de forma controlada el pipeline y revisar el champion resultante:

Capa	Qué hay que adaptar
Pregunta y estimando	Definir unidad, `D`, `Y`, población, ATE/ATT y decisión antes de mirar resultados; documentarla en `docs/`, no solo en código.
Datos y autorización	Asegurar permiso, minimización y covariables pre-tratamiento; crear particiones y hashes sin filtrar resultados potenciales al ajuste.
Esquema y DAG	Actualizar roles en `datos.py`, metadatos, `identification.json` y DAG; justificar el ajuste. No basta copiar las 25 variables IHDP.
Estimación y falsificación	Revisar modelos auxiliares, soporte, balance, comparadores, sensibilidad y placebos apropiados al resultado y al diseño.
Servicio y tarjeta	Actualizar estrato/covariables del perfil, textos, unidades, rangos, restricciones, limitaciones y pruebas de contrato.
Monitor y registro	Justificar ventanas y umbrales, probar puerta, crear nueva versión y promover solo con la política correspondiente.

La interfaz JSON puede reutilizarse, pero el DAG, la identificación y la evidencia no se heredan a otro dominio. Para no perder trazabilidad, conserva una copia del champion anterior y nunca edites silenciosamente los JSON de una versión inmutable.

10 Puerta, abstención e integridad

10.1 Qué bloquea realmente una respuesta

El servicio verifica alias y hashes de manifiesto, estimación y tarjeta al cargar. La puerta activa se toma del campo superior `estado_puerta` de `artifacts/monitor_estado.json` si su versión coincide con el champion; si no, se usa `manifest.puerta`. El `peor_estado_puerta` de las cinco ventanas no sustituye automáticamente al campo superior.

Acción del monitor	Estado API	Conducta
`continuar`	`pasa`	Devuelve estimación si perfil e integridad son válidos
`alertar`	`alerta`	Devuelve estimación con el estado visible; requiere revisión
`reentrenar`	`reentrenar`	No bloquea por sí solo; señala deriva de covariables
`revisar_identificacion`	`revisar_identificacion`	No bloquea por sí solo; señala placebo/control negativo que exige revisión
`abstenerse`	`falla`	Abstención sin estimación

Además de la puerta `falla`, hay abstención si la integridad no verifica, faltan campos necesarios o un perfil sale del rango de entrenamiento. El API devuelve normalmente HTTP 200, `estado = abstencion`, `motivo`, versión y puerta, sin `efecto` ni `proyeccion`. Los estados `alerta`, `reentrenar` y `revisar_identificacion` no añaden automáticamente una lista de advertencias: el consumidor debe leer `estado_puerta`.

10.2 Probar la abstención sin modificar artefactos

Con el champion conservado, `bw = 99` está fuera del rango:

```
curl -sS -i -X POST http://127.0.0.1:8000/consulta \
  -H 'Content-Type: application/json' \
  -d '{"bw":99}'
```

Se espera HTTP 200 y un cuerpo semejante a:

```
{
  "estado": "abstencion",
  "tipo": "estrato",
  "motivo": "bw=99 fuera de [-3.1332, 3.0532]",
  "estado_puerta": "pasa",
  "version": "v0009"
}
```

Que `estado_puerta = pasa` no contradice la abstención: la puerta del bundle está sana, pero ese perfil no está soportado. No devuelvas 7,2 por ser «el estrato más cercano». `humo.py` prueba además, en copias temporales, alias ausente, estimación o monitor corruptos y manifiesto ausente. En esos fallos `/salud`, `/efecto`, `/consulta` y `/heterogeneidad` responden con abstención y puerta `falla`, sin `efecto` ni `proyeccion`; el formulario sigue disponible con un aviso. No edita los artefactos guardados.

11 Monitor y regla operativa

11.1 Ventanas conservadas

La ventana de producción es `sana`; las demás son pruebas de estrés que muestran la regla, no cinco estados simultáneos del servicio actual.

Ventana	PSI máx	SMD pond.	ATE ventana	Acción
`sana`	0,024	0,07	5,64	`continuar` / `pasa`
`deriva`	0,883	0,03	5,16	`reentrenar`
`positividad`	3,069	5,16	7,2	`abstenerse` / `falla`
`balance`	0,029	0,05	3,63	`alertar`
`confusor_oculto`	0,0	0,03	1,0	`alertar`

La ventana sana difiere del ATE champion en 0,49, por debajo del umbral de estabilidad 1,25. La ventana de positividad rompe el solapamiento y obliga a abstenerse. Balance y confusor oculto se detectan por estabilidad del efecto. El confusor oculto es una prueba de estrés: el monitor no observa `U`; la sensibilidad es quien explora su fuerza.

Los umbrales `smd_max = 0,50`, `smd_ponderado_max = 0,25`, `psi_alerta = 0,25`, `solape_min = 0,30` y `efecto_estabilidad = 0,20` son **convenciones iniciales**, no límites universales. La variación ±20 % no cambió acciones en esta corrida (0 cambios): eso es estabilidad local, no validación externa ni calibración definitiva.

11.2 Tarjeta, manifiesto y champion

`tarjeta.json` recoge estimando, población, supuestos, umbrales, semilla, hash del dataset, estimador, uso previsto, limitaciones, soporte, puerta, sensibilidad, placebo e identificación. Cada registro crea `artifacts/registro/vNNNN/` con estimación, tarjeta y `manifest.json` con hashes. `alias_champion.json` selecciona la versión; `registro/champion` se conserva por compatibilidad.

`promover()` bloquea una versión cuyo `manifest.puerta` existente sea `falla`; una en `alerta` exige aceptación explícita. El flujo general registro → monitor **no es transaccional**: la promoción puede ocurrir antes del monitor y no hay rollback automático si una etapa posterior falla. En ese caso el servicio puede bloquear respuestas, pero la promoción no se deshace sola.

`v0001`–`v0008` se conservan intactos y el champion de este paquete es `v0009`. No edites un JSON versionado para «corregir» una cifra: vuelve a estimar y registra otra versión.

12 Evidencia, mantenimiento y regeneración

12.1 Consultar la evidencia existente

Pregunta	Dónde comprobarla
Esquema, splits y pared de verdad	`evidencia/01_datos.txt`
ATE, OLS, comparación y positividad	`evidencia/02_estimacion.txt`
E-value, valor de robustez y Manski	`evidencia/03_sensibilidad.txt`
Placebos	`evidencia/04_placebo.txt`
Puerta, ventanas y umbrales ±20 %	`evidencia/05_puerta.txt`
Versión, alias y manifiesto	`evidencia/06_registro.txt`
Contratos de servicio y abstenciones	`evidencia/07_servicio.txt`
Hashes de DAG y módulos	`evidencia/08_dag.txt`
Pruebas	`evidencia/09_pruebas.txt`
Cobertura en DGP ilustrativo propio	`evidencia/10_cobertura.txt` y `.json`

`README.md` resume la operación; `AUDITORIA.md` enlaza el código publicado; `docs/pregunta_ihdp.md` conserva la pregunta; `artifacts/registro/v0009/` es la fuente del champion. Si una frase del manual y una respuesta difieren, comprueba primero versión y artefactos: no presentes como actual una cifra de otra versión.

12.2 Regenerar deliberadamente

Con el entorno activo y desde la raíz:

```
export SIC_ESCENARIO=synthetic/ihdp
make demo
make pruebas
```

El flujo `demo` recorre **verificación de los CSV incluidos** → estimación → registro → monitor → humo; no genera IHDP desde una semilla. `cobertura.py` usa otro DGP ilustrativo. **Demo crea una nueva versión inmutable y puede promover un nuevo champion.** `make evidencia` además reejecuta la cadena y sobrescribe salidas generadas y hashes de `evidencia/`. No lo uses como comprobación de lectura ni como instalación.

Registro y monitor no forman una transacción ni revierten automáticamente una promoción si algo falla después. Tras regenerar, comprueba `artifacts/alias_champion.json`, `artifacts/monitor_estado.json` y `/salud` antes de usar el servicio.

Otros comandos de mantenimiento:

```
make cobertura    # escribe cobertura del DGP ilustrativo propio
make dag          # regenera SVG; requiere Graphviz dot
make cierre       # compila este manual Typst a PDF
make limpiar      # PELIGRO: borra TODO artifacts/, incluido champion
```

No ejecutes `make limpiar` como limpieza rutinaria. Cachés y `.venv` son regenerables; `artifacts/` es el producto versionado. Para actualizar el manual sin tocar el modelo basta compilar el `.typ`; no hace falta ejecutar `demo`, `evidencia` ni promover versiones.

13 Operación, seguridad y alcance

Aspecto	Regla de uso
Red	Los scripts y Compose vinculan a `127.0.0.1`. No expongas el puerto directamente a Internet: no hay autenticación, autorización, TLS, rate limiting ni auditoría de acceso.

Aspecto	Regla de uso
Despliegue real	Requeriría proxy/TLS, identidad, límites, gestión de secretos, política de logs, pruebas en el sistema destino y supervisión operativa. No lo confundas con local.
Serialización e integridad	Los bundles son JSON, no `pickle` ni `joblib`. Los hashes detectan diferencias, pero no autentican origen ni impiden una sustitución coordinada de archivos y manifiesto. Usa una procedencia confiable; no aceptes bundles arbitrarios.
Datos	Los datos incluidos son sintéticos. Un caso real exige autorización y controles de privacidad; no envíes perfiles personales a un servicio sin gobernanza.
Actualizaciones	Reinicia para recargar champion o monitor. Revisa `/salud`, ejecuta pruebas y conserva la versión anterior para rollback deliberado.
Decisiones	No usar como recomendación clínica, laboral o de política pública. Los outputs son una demostración sobre simulación y dependen de supuestos no totalmente verificables.

13.1 Validación de esta segunda auditoría

Esta tabla separa lo ejecutado de lo solo revisado. No es una certificación de despliegue ni una sustitución de las pruebas en el equipo de cada usuario.

Plataforma	Verificación realizada	Alcance del resultado
macOS arm64	Copia limpia en ruta con espacios; `setup.sh` dos veces; `datos.py`, pruebas y humo; `arrancar.sh`, healthcheck y consultas HTTP reales	Python 3.13.14; 140 paquetes; `uv pip check`, pruebas y humo OK. ATE 5,2 [3,9; 6,4], estrato y abstención fuera de soporte comprobados
Linux arm64, Docker Engine mediante Colima	`docker build`, `docker run`, healthcheck repetido y endpoints HTTP	Contenedor `healthy`; ATE, estrato y abstención correctos
Docker Compose	Revisión estática de `docker-compose.yml`	No ejecutado: el plugin Compose no estaba instalado
Windows / lanzador gráfico	Compilación MSVC y `ctest --self-test` en `windows-2022`, ejecución Actions 43322853322	Ambas superadas con `v0008`. Publicar exige también instalación y arranque `--smoke-test`, integridad, puerta `pasa` y `/efecto` válidos; los assets no se adjuntan si falla. Falta comprobar la interacción gráfica en el PC del usuario y repetir la prueba integral con `v0009`
Windows / PowerShell	Revisión estática de los `.ps1` y comandos, compatibles con sintaxis 5.1	Scripts no ejecutados en Windows nativo; el lanzador recomendado no depende de ellos

Las pruebas de `humo.py` también comprobaron fallo cerrado ante alias/manifiesto ausentes y JSON de estimación/monitor corruptos, usando copias temporales. La auditoría no ejecutó `make demo` ni `make evidencia` y mantuvo los hashes de champion y artefactos conservados. Persisten límites de reproducibilidad: no hay lockfile de dependencias transitivas ni digest fijado para la imagen base Docker. Los scripts PowerShell, la GUI en el PC del usuario, Compose, otras arquitecturas y una exposición real a Internet no quedan certificados por estas pruebas.

14 Interpretación final para lectores no técnicos

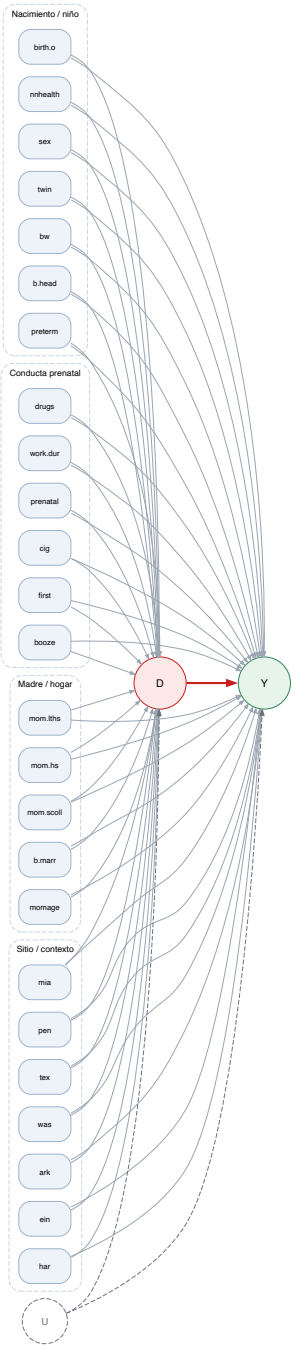
Resultado. En el escenario sintético y para la población de entrenamiento, el ATE AIPW es 5,2 puntos del test cognitivo, con IC95 [3,9; 6,4]. Es compatible con un beneficio promedio bajo el DAG y los supuestos declarados. No significa que todos mejoren 5,2, que un perfil concreto vaya a mejorar esa cantidad ni que el programa real tenga ese efecto.

Qué aporta el sistema. Ajusta por 25 covariables, separa ajuste y predicción auxiliar, mide positividad, compara estimadores, registra una versión con hashes, revisa sensibilidad y placebos, vigila ventanas y se abstiene cuando su contrato no permite responder. La infraestructura hace explícitos los límites; no convierte un supuesto incumplido en un resultado válido.

Qué exigir antes de decidir. Una versión e integridad correctas; una puerta comprendida; soporte y unidades acompañando al número; sensibilidad y placebos correctamente descritos; estabilidad fuera de `train`; y un análisis de costes, riesgos, validez externa y alternativas. Las cotas de Manski incluyen cero: sin ignorabilidad, la simulación no sostiene por sí sola una conclusión puntual positiva.

Cuándo no mostrar un número. Integridad rota, puerta `falla`, perfil fuera de rango o entrada incompleta. Cuándo revisar antes de confiar: deriva, fallo de placebo/control negativo, pérdida de solapamiento, cambios de población o estimaciones incompatibles entre particiones. En ninguno de esos casos corresponde imputar el efecto anterior.

15 DAG completo de `synthetic/ihdp`



DAG (`dag_ihdp_detalle.svg`): un nodo por covariable pre-tratamiento (25), tratamiento `D = treatment` y resultado `Y = outcome`; aristas según el DGP declarado. `U` es una amenaza hipotética de confusión no medida, no una variable documentada en el DGP del benchmark.

Grupos de covariables. Nacimiento / niño: `bw`, `b.head`, `preterm`, `birth.o`, `nnhealth`, `sex`, `twin`. Conducta prenatal: `cig`, `first`, `booze`, `drugs`, `work.dur`, `prenatal`. Madre / hogar: `momage`, `mom.lths`, `mom.hs`, `mom.scoll`, `b.marr`. Sitio / contexto: `ark`, `ein`, `har`, `mia`, `pen`, `tex`, `was`.

La versión agrupada está en `figuras/dag_ihdp_plate.svg`; la del escenario `synthetic/jobs`, en `figuras/dag_jobs.svg`. El grafo de módulos y el DAG causal no deben confundirse: uno describe imports de software; el otro declara relaciones causales.

16 Reutilización, defensa y rúbrica

16.1 Decisiones propias y preparación de la defensa

Se reutilizan AIPW con cross-fitting, tarjeta, registro, puerta y servicio. Se adaptan las covariables, el corte `bw`, los umbrales, las ventanas y la capa de sensibilidad. Las decisiones propias incluyen el corte en la mediana de `train`, ventanas de deriva, positividad, balance y confusor oculto, y las medidas apropiadas a resultado continuo. El caso sigue siendo guiado: no se presenta como trabajo final autónomo.

Para una defensa de 15 minutos (10 de exposición y 5 de preguntas):

1. Muestra `monitor_estado.json` y `evidencia/05_puerta.txt` para explicar puerta y abstención; complementa con `evidencia/07_servicio.txt`.
2. Explica ignorabilidad como supuesto no observable acotado por E-value 2,07 (IC 1,85), RV OLS 37,4 % y Rosenbaum Γ 2,21 para su contraste emparejado; nunca como supuesto demostrado ni como una única medida del AIPW.
3. Indica qué haría retirar la recomendación: confusión suficiente, fallo de placebo, ruptura de solapamiento o deriva clasificada como reentrenar/abstenerse. Si no puedes mostrar el artefacto o reproducir la cifra, no la presentes como evidencia propia.

16.2 Estado frente a los criterios 1–6

ID	Subcriterio	Estado	Evidencia / límite
1.1	Pregunta de decisión y estimando	Parcial	Secciones 4 y 14; pregunta y ATE declarados; el caso es guiado
1.2	Datos y autorización	Completo	Sección 4; hashes de splits; datos sintéticos sin registros personales
2.1	Estrategia de identificación	Completo	Sección 5: backdoor por observables y límites de DoWhy
2.2	Supuestos clasificados	Completo	Secciones 5–7 y tarjeta: observable/no observable y evidencia
2.3	Estimación con intervalo y validación	Completo	Sección 6: AIPW K=5, IC, comparadores, particiones y cobertura en DGP propio
2.4	Interpretación y sensibilidad	Completo	Sección 7: E-value, RV, Rosenbaum Γ, Manski y límites
3.1	Servicio y contrato	Completo	Secciones 8–10: endpoints, `proyeccion`, IC condicional, alias y abstención; `humo.py`
3.2	Model card completa y propia	Completo	`tarjeta.json`: uso previsto, limitaciones, soporte y puerta
3.3	Registro con manifiesto	Completo	Champion `v0009`, versiones y bloqueo de `falla`; registro → monitor no transaccional
4.1	Señales con umbral y acción	Completo	Sección 11: PSI, SMD, ESS, solape, estabilidad y sensibilidad local ±20 %
4.2	Regla de decisión probada	Completo	Secciones 10–11: cinco acciones, ventanas y pruebas de humo
4.3	Control negativo o placebo	Completo	Sección 7: placebos de tratamiento y resultado con lectura y límites
4.4	Supuesto no observable	Completo	Secciones 5–7: ignorabilidad declarada y acotada, no demostrada
5.1	Cadena coherente	Completo	Figuras iniciales y sección 12: datos, estimación, registro, puerta y servicio
5.2	Paquete de evidencia	Completo	Sección 12 y `evidencia/`: comando, salida y hash por cifra

ID	Subcriterio	Estado	Evidencia / límite
5.3	Reutilización de lo propio	Completo	Sección 9: adaptación de datos/contratos y esta sección
6.1	Decisiones propias	Completo	Corte `bw`, umbrales, ventanas, sensibilidad y decisiones de servicio
6.2	Defensa en vivo	Pendiente	Guion y preguntas preparados; defensa y evaluación oral no realizadas
6.3	Documento de cierre	Completo	Sección 14: efecto, decisión, supuestos y cuándo dejar de confiar

Lectura. Este manual describe el estado del prototipo, no una nota otorgada. La referencia académica completa es `ENTREGABLE_FINAL_GUIA_Y_RUBRICA.typ` y su PDF. El subcriterio 1.1 se mantiene en parcial por ser caso guiado y 6.2 queda pendiente hasta la defensa oral. La auditoría técnica no equivale a una calificación: la nota final requiere evaluación humana y defensa de la evidencia. La ejecución nativa en Windows sigue sin verificarse.